

Cognizant NPN Walmart M5 Dataset & Problem Statement

Deep, file-by-file and column-by-column explanation based on the
uploaded CSV files

Files covered in this document

1. sales_train_validation.csv
2. sales_train_evaluation.csv
3. calendar.csv
4. sell_prices.csv
5. sample_submission.csv

Important note

Yes. I inspected the actual five CSV files you uploaded, so the explanation below is based on your exact dataset, not a generic M5 description.

1. Understanding the problem statement

Your Cognizant problem statement is essentially asking you to build a retail demand forecasting system.

The statement says:

Retail Demand Forecasting (Walmart M5, Hierarchical)

and asks you to:

forecast item-level demand across stores and product hierarchies while considering stockouts, promotions, holidays and intermittent demand, and produce accurate 28-day-ahead forecasts.

Let's break every part down.

A. What is "Retail Demand Forecasting"?

Suppose Walmart sells a product:

HOBBIES_1_001

at:

CA_1

The store might have historical sales like:

Date	Units sold
Monday	5
Tuesday	7
Wednesday	3
Thursday	0
Friday	8
Saturday	10
Sunday	6

That is demand forecasting.

So your model takes:

Past sales + product information + store information + calendar + price + events

and produces:

future sales predictions

B. Why does Walmart need this?

Imagine Walmart predicts:

Actual demand = 100 units

but Walmart stocks:

60 units

Then:

40 customers may not get the product → stockout

On the other hand, if Walmart stocks:

200 units

and only sells:

100

then Walmart has:

100 units of excess inventory

That can lead to:

- unnecessary inventory cost
- storage problems
- wastage for some products
- inefficient capital use

So accurate forecasting helps Walmart decide:

How much should we stock?

C. What does "item-level demand" mean?

This is very important.

We are not just predicting Walmart's total sales.

We are predicting sales for a specific item at a specific store.

```
HOBBIES_1_001 + CA_1  
HOBBIES_1_001 + CA_2  
HOBBIES_1_001 + CA_3
```

These are different forecasting series.

So your target becomes:

How many units of item X will store Y sell on each of the next 28 days?

D. What does "across stores" mean?

Your dataset contains 10 stores:

CA_1
CA_2
CA_3
CA_4

TX_1
TX_2
TX_3

WI_1
WI_2
WI_3

These belong to three states:

CA → California
TX → Texas
WI → Wisconsin

A particular product could sell:

CA_1 → 10 units
CA_2 → 5 units
CA_3 → 8 units

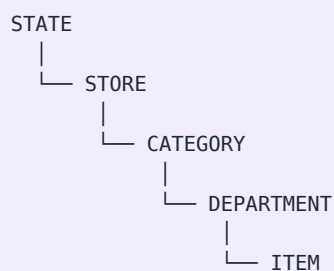
So the model needs to understand that different stores have different demand patterns.

E. What does "hierarchical" mean?

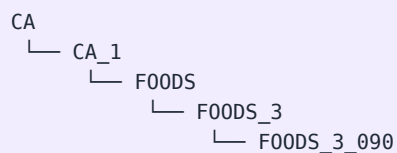
This is one of the most important parts of your problem.

Your data has multiple levels of organization.

Think of it like:



For example:



So an item belongs to a department.

A department belongs to a category.

A store belongs to a state.

This is called a hierarchy.

F. Why does hierarchy matter?

Suppose:

```
F00DS
  ↓
F00DS_3
  ↓
F00DS_3_090
```

If the individual item forecast says:

100 units

but the department-level information strongly suggests only:

50 units

then there is a consistency problem.

A good hierarchical forecasting solution tries to produce forecasts that make sense across the different aggregation levels.

For example:

```
Item A = 30
Item B = 20
Item C = 50
-----
Department total = 100
```

That is logically consistent.

G. What are "external covariates"?

A covariate is an additional variable that can help predict demand.

Your problem specifically mentions:

Price

How expensive the product is.

Promotion

Whether the product may be receiving promotional treatment indirectly reflected through price changes/events.

Holiday/Event

Special events can change customer behavior.

There are also other useful variables in your calendar dataset:

- weekday
- month
- year
- SNAP indicators

These are not the target itself. They are information that helps explain the target.

H. What is intermittent demand?

This is another very important concept in your project.

Some products are not sold every day.

For example:

Monday	0
Tuesday	0
Wednesday	0
Thursday	0
Friday	3
Saturday	0
Sunday	0

This is called intermittent or sparse demand.

The model cannot assume:

"If yesterday sales = 0, tomorrow sales will also be 0."

There may be long periods of zero sales followed by sudden purchases.

That's why this dataset is more challenging than a simple smooth time series.

I. What does "28-day-ahead forecasting" mean?

This is the final task.

Suppose the last known historical date is:

April 24, 2016

Your model must predict:

Day 1
Day 2
Day 3
...
Day 28

So the output is 28 values for each series.

For example:

Then the model predicts future daily demand for the next 28 days.

J. What exactly does Cognizant want you to build?

In simple words:

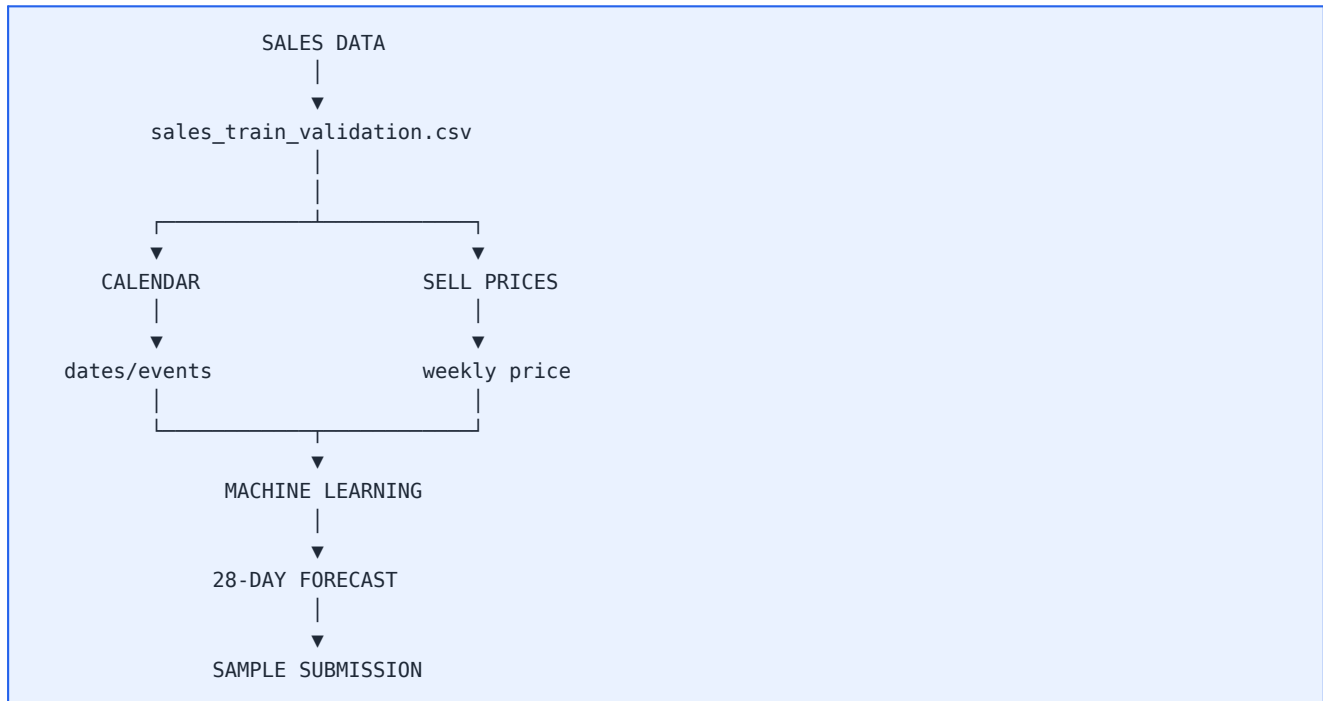
Build an ML/time-series system that learns from Walmart's historical sales and related information, then predicts how many units each item each store will sell for the next 28 days, while respecting the retail hierarchy and using price, promotions, holidays and other temporal information.

That's the entire project in one sentence.

2. Now let's understand YOUR dataset

Your five files play different roles.

Think of the whole dataset like this:



Now let's go file by file.

□ **FILE 1** — sales_train_validation.csv

This is your main historical sales dataset.

Your uploaded file has:

- 30,490 rows
- 1,919 columns

The first 6 columns identify the product/store hierarchy.

Then you have:

d_1
d_2
d_3
...
d_1913

which represent daily sales.

Column 1 — id

Example:

HOBBIES_1_001_CA_1_validation

This is the unique identifier for one forecasting series.

You can think of it as:

"This exact product at this exact store."

Let's split the example:

```
HOBBIES_1_001_CA_1_validation
|      |      |      |      |
|      |      |      |      | dataset/series suffix
|      |      |      |      | store
|      |      |      |      | item
|      |      |      |      | department
|      |      |      |      | category
```

The important practical idea is:

id identifies a particular item-store combination.

Column 2 — item_id

Example:

```
HOBBIES_1_001
```

This identifies the specific product/item.

Your dataset contains:

3,049 unique items

For example:

```
HOBBIES_1_001
HOBBIES_1_002
FOODS_3_090
...
```

So:

item_id answers: Which product is this?

Column 3 — dept_id

Example:

```
HOBBIES_1
```

This tells you the department that the item belongs to.

Your dataset has 7 departments:

```
HOBBIES_1
HOBBIES_2

HOUSEHOLD_1
HOUSEHOLD_2

FOODS_1
FOODS_2
FOODS_3
```

So:

dept_id answers: Which department does this item belong to?

Column 4 — cat_id

Example:

HOBBIES

This is the broader product category.

Your dataset has 3 categories:

HOBBIES
HOUSEHOLD
FOODS

So the hierarchy can look like:

HOBBIES
↓
HOBBIES_1
↓
HOBBIES_1_001

Therefore:

- `cat\_id` = broader category
- `dept\_id` = department
- `item\_id` = specific product

Column 5 — store_id

Example:

CA_1

This tells us which Walmart store the sales came from.

Your dataset has 10 stores:

CA_1
CA_2
CA_3
CA_4

TX_1
TX_2
TX_3

WI_1
WI_2
WI_3

So:

store_id answers: Where was this item sold?

Column 6 — state_id

Example:

CA

This is the state containing the store.

Your dataset has:

CA
TX
WI

Relationship:

CA
├─ CA_1
├─ CA_2
├─ CA_3
└─ CA_4

and:

TX
├─ TX_1
├─ TX_2
└─ TX_3

So:

state_id answers: Which state is this store in?

Columns 7 onward — d_1, d_2, ..., d_1913

These are the actual daily sales quantities.

For example:

```
d_1 = 0
d_2 = 1
d_3 = 3
d_4 = 2
...
```

means that particular item/store combination sold those quantities on those days.

IMPORTANT

`d\_1` does not mean "day of the month 1".

It is simply an internal day identifier.

The actual date comes from:

calendar.csv

Mapping d_1 to an actual date

In your uploaded calendar.csv:

```
d_1 → 2011-01-29
d_2 → 2011-01-30
d_3 → 2011-01-31
d_4 → 2011-02-01
...
```

So:

```
sales_train_validation.csv
d_1 = 5
```

and:

```
calendar.csv
d_1 = 2011-01-29
```

together mean:

This item sold 5 units on January 29, 2011.

This connection is extremely important.

Why are there 1,913 d_ columns?

Your validation sales file goes from:

```
d_1
```

through:

```
d_1913
```

And in your calendar:

```
d_1913 = 2016-04-24
```

So the validation dataset contains daily sales history from:

January 29, 2011 → April 24, 2016

FILE 2 — sales_train_evaluation.csv

This looks very similar to the validation file, but there is an important difference.

It has:

- 30,490 rows
- 1,947 columns

The first 6 columns are the same:

```
id
item_id
dept_id
cat_id
store_id
state_id
```

Then it goes:

```
d_1
...
d_1941
```

Why does evaluation have more days?

Because:

Validation:

d_1 → d_1913

Evaluation:

d_1 → d_1941

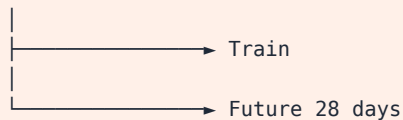
We confirmed:

d_1913 = 2016-04-24
d_1941 = 2016-05-22

So the evaluation dataset contains an additional 28 days of sales.

That is directly related to the 28-day forecasting problem.

Historical sales



The additional 28 days help evaluate predictions against actual sales in the original competition setup.

□ FILE 3 — calendar.csv

This file contains date-related and event-related information.

Your file has:

- 1,969 rows
- 14 columns

These 14 columns are:

```
date
wm_yr_wk
weekday
wday
month
year
d
event_name_1
event_type_1
event_name_2
event_type_2
snap_CA
snap_TX
snap_WI
```

Column 1 — date

```
2011-01-29
```

This is the actual calendar date.

Unlike d_1, this is a human-readable date.

```
d_1 → 2011-01-29
d_2 → 2011-01-30
d_3 → 2011-01-31
```

We'll use it to create features such as:

- day
- week
- month
- year
- weekend

Column 2 — wm_yr_wk

```
11101
```

This is the Walmart-specific year-week identifier.

It allows us to connect the calendar to the weekly prices in `sell_prices.csv`.

```
calendar.csv
wm_yr_wk
|
▼
sell_prices.csv
wm_yr_wk
```

This is extremely important because prices are stored by week, not by individual day.

Column 3 — weekday

Saturday
Sunday
Monday
Tuesday
...

This tells us the name of the day of the week.

Demand can behave differently on:
- weekdays
- weekends

For example, a product may sell more on Saturday than Monday.

Column 4 — wday

Saturday → 1
Sunday → 2
Monday → 3
Tuesday → 4
...

This is the numeric representation of weekday.

So it provides a numerical version of weekday.

Column 5 — month

1
2
3
...
12

This tells us the month.

Useful because demand can be seasonal.

December → Christmas shopping

could change demand.

Column 6 — year

2011
2012
2013
...
2016

This tells us the year.

It allows the model to learn longer-term trends.

Column 7 — d

d_1
d_2
d_3

This is the day identifier.

This column is what connects the calendar file to:

```
sales_train_validation.csv
```

For example:

```
calendar.d = d_100
```

corresponds to:

```
sales.d_100
```

So it connects the date to the sales row.

Column 8 — event_name_1

```
SuperBowl
ValentinesDay
NBAFinalsEnd
Christmas
Thanksgiving
```

This contains the name of a major event/holiday when applicable.

Many rows are empty because there isn't an event every day.

This can influence consumer behavior.

Column 9 — event_type_1

```
Holiday
Sporting
Cultural
Religious
```

This tells you the type of event associated with event_name_1.

The exact value depends on the event.

```
event_name_1 = NBAFinalsEnd
event_type_1 = Sporting
```

means the first event is related to a sporting event.

Column 10 — event_name_2

There can occasionally be a second event on the same date.

This column records that second event.

Often it is empty.

Column 11 — event_type_2

The type/category of the second event.

Again, often empty.

So:

```
event_name_2
and
event_type_2
```

are basically the second event slot.

Column 12 — snap_CA

0 = SNAP not active
1 = SNAP active

This is a SNAP indicator for California.

SNAP is the Supplemental Nutrition Assistance Program.

So for products/stores in California, SNAP timing can potentially influence purchasing behavior.

Column 13 — snap_TX

0 = not active
1 = active

Same concept, but for:

Texas

Column 14 — snap_WI

Same concept for Wisconsin

0 = not active
1 = active

FILE 4 — sell_prices.csv

This is your price information.

Your uploaded file has:

- 6,841,121 rows
- 4 columns

The columns are:

store_id
item_id
wm_yr_wk
sell_price

Column 1 — store_id

CA_1

Which store is this price associated with?

Column 2 — item_id

HOBBIES_1_001

Which item is this price associated with?

Column 3 — wm_yr_wk


```
11325
```

This is the Walmart week identifier.

This is important because prices are stored by week, not by individual day.

```
calendar.wm_yr_wk
      ↓
sell_prices.wm_yr_wk
```

acts as a bridge.

Column 4 — sell_price

```
9.58
8.26
```

This is the selling price of that item at that store during that Walmart week.

For example, your first few rows show:

```
CA_1
HOBBIES_1_001
11325
9.58
```

and later:

```
CA_1
HOBBIES_1_001
11327
8.26
```

So the price can change over time.

That price change can provide useful information for forecasting demand.

□ How sell_prices.csv connects to sales

This is critical.

Suppose we have:

```
Sales:
item = HOBBIES_1_001
store = CA_1
date = 2011-01-29
sales = 3
```

From calendar.csv:

```
2011-01-29
↓
wm_yr_wk = 11101
```

Then we can search sell_prices.csv for:

```
store_id = CA_1
item_id = HOBBIES_1_001
wm_yr_wk = 11101
```

and get the corresponding price.

So the three files work together:

SALES
+
CALENDAR
+
PRICE

□ FILE 5 — sample_submission.csv

This is different from the other files.

It doesn't provide historical information.

It tells us:

What format should our predictions have?

Your sample file has:

- 60,980 rows
- 29 columns

Columns:

id
F1
F2
F3
...
F28

Column 1 — id

Example:

HOBBIES_1_001_CA_1_validation

This identifies the series for which we are predicting.

Columns F1 through F28

These are our forecast periods.

Think:

F1 = forecast day 1
F2 = forecast day 2
F3 = forecast day 3
...
F28 = forecast day 28

So if the model predicts:

```
F1 = 5  
F2 = 7  
F3 = 4  
...  
F28 = 6
```

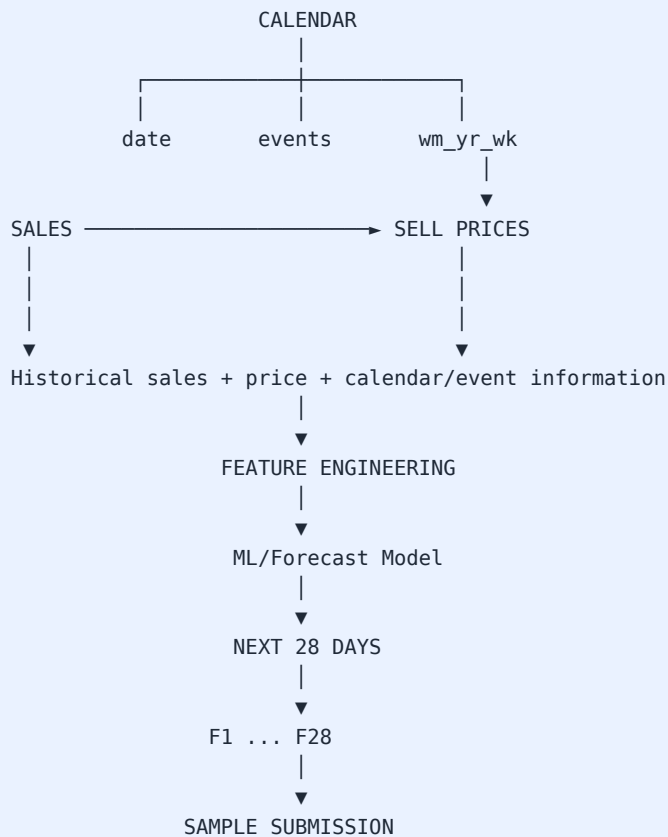
that means:

predicted demand for each of the next 28 days.

This directly matches your Cognizant problem statement.

The complete relationship among your files

This is the most important thing you should understand before coding.



□ One simple example connecting EVERYTHING

Suppose we take:

```

item_id = HOBBIES_1_001
store_id = CA_1

```

In `sales_train_validation.csv`, we might have:

```

d_1 = 1
d_2 = 3
d_3 = 2
...

```

From `calendar.csv`:

```

d_1
↓
2011-01-29
Saturday
January
2011
wm_yr_wk = 11101

```

From `sell_prices.csv`:

```
CA_1
HOBBIES_1_001
11101
price = ...
```

Now the model can learn:

On Saturdays, when the item has this price, at this store, with this historical demand pattern, what is the likely future demand?

Then:

MODEL



28-day forecast

```
F1  = 4
F2  = 5
F3  = 3
...
F28 = 6
```

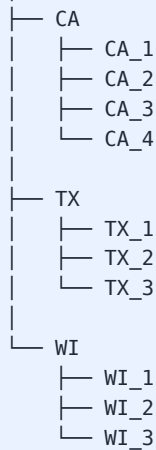
□ Your dataset in one table

File	What it means	Main purpose
`sales_train_validation.csv`	Historical daily sales	Train the forecasting model
`sales_train_evaluation.csv`	Historical sales extended by 28 days	Evaluation/competition setup
`calendar.csv`	Dates, weekdays, events, SNAP	Time/event features
`sell_prices.csv`	Weekly item/store prices	Price-related features
`sample_submission.csv`	Required prediction structure	Final 28-day output format

□ The hierarchy in YOUR actual data

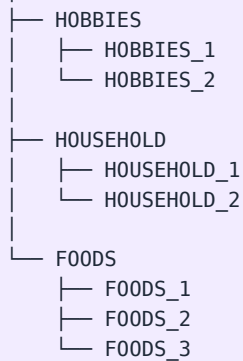
Your uploaded data gives us this structure:

3 States



Product hierarchy:

3 Categories



Then below those departments are:

3,049 individual items

And there are:

30,490 item-store series

because the 3,049 items are observed across the 10 stores.

□ One distinction you should remember for your presentation

Don't say:

"We have 3,049 rows of products."

Say:

"The dataset contains 3,049 unique products and 10 stores, producing 30,490 item-store forecasting series."

That's a much more technically accurate description of the data you uploaded.

□ The most important columns to remember for the hackathon

You don't need to memorize 1,913 individual day columns.

Conceptually, remember:

Product information

item_id
dept_id
cat_id

Location information

store_id
state_id

Sales target

d_1 ... d_1913 / d_1941

Time information

date
weekday
wday
month
year

Event information

event_name_1
event_type_1
event_name_2
event_type_2

SNAP

snap_CA
snap_TX
snap_WI

Price

sell_price

Forecast output

F1 ... F28

□ The most important thing to remember

The project isn't simply:

"Train a model on sales."

It is:

```
Historical sales
+
Product hierarchy
+
Store hierarchy
+
Calendar patterns
+
Events/Holidays
+
Price
+
Sparse/zero demand
↓
Forecasting Model
↓
28-day predictions
```

That is why this is a good hackathon problem.

Your next step should be EDA, not model training.

We should take your actual uploaded data and investigate questions like:

- How many zero-sales observations are there?
- Which category sells the most?
- Which stores sell the most?
- Which products are highly intermittent?
- How does demand change by weekday?
- What happens during holidays?
- How does price relate to sales?
- Which products/stores have the strongest trends?

That analysis will tell us what kind of forecasting approach we should build, rather than blindly choosing a model.